

Lists in Python

List is a collection of heterogeneous data arranged in a sequence. The values can be of any data type like string, integer, float, objects or even a list. Each value in a list is called an element and its position in a list is referred by an index number. The number of elements in a list will make the length of a list. All the elements in a list are separated by comma and enclosed in square brackets [ ].

List is a mutable data type in Python as the content of the list can be added, modified or deleted by the user anytime throughout the code. A number of operations can be performed on the list by using a few built-in list functions which we will study in detail in this chapter.

Important Features of Lists

Based on the above definition, the list has the following important features:

- It is mutable data type.
- It is an ordered sequence of values.
- Each element is separated by comma and enclosed in square brackets [ ].
- Each value/element is accessed by an index number.
- The values can be added, modified or deleted by the user throughout the program.
- It stores the values of different data types.

Creation of List

A list is created by enclosing the values within square brackets. Each value/element is separated by a comma and stored under a common name. These values can be of different data types. The name follows the rules of naming conventions for identifiers as shown below:

Examples	Commands
Creating an empty list	`list1 = []`
Creating a list with single value	`list2 = [56]`
Creating a list of friends	`friends = ["Saisha", "Advika", "Arshia"]`
Creating a list of marks	`marks=[92, 91, 89, 95, 93]`
Creating a list of students with roll no and percentage (list of mixed values)	`students = [21, "Gunjan", 94.5, 34, "Rohit", 91.3]`
Creating a list within another list	`section = ["A", 25, "English", ["B",34], ["C",37], 56.7]`
Creating a list with repeated values	`L1 = [10] * 5  # L1 will have [10, 10, 10, 10, 10]`
Creating a list from another sequence or string using list() function	`alpha = list("hello") #string converted to a list #output will ['h', 'e', 'l', 'l', 'o']`
Values input from the user in a list	`L1 = list(input("enter numbers :"))`

Access Elements of a List

The elements of a list can be accessed by using its index number starts with 0 (zero). There are two different ways of using the index number:

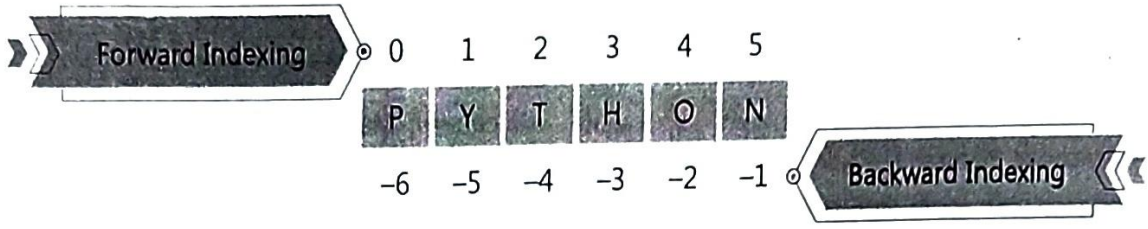
- **Forward Indexing:** The index number begins with 0 till length-1 and we refer the elements from left to right.



, **Negative/Backward Indexing:** The index number begins with -1 till -length and we refer the elements from right to left.

For example,

L1 = ['P', 'Y', 'T', 'H', 'O', 'N']



Each element can be accessed by using:

listname[index number]

Commands	Output
`print(L1[0])`	P
`print(L1[3])`	H
`print(L1[-3])`	H
`print(L1[-1])`	N
`print(L1)  #print complete list`	['P', 'Y', 'T', 'H', 'O', 'N']
`print(L1[0], L1[4])`	P O

Task

#Coding & Computational Thinking

If word = ['E', 'D', 'U', 'C', 'A', 'T', 'I', 'O', 'N']

What will be the output of the following instructions:

- print(word[:1:-2])
- print(word[-1:])
- print(word[:-1])

List Operations

A list is a collection of items that can be manipulated using various operations. These operations involve adding, removing, and changing elements within a list. Learning about list operations helps in organizing and processing data effectively, which is essential for basic programming tasks and problem-solving.

Adding New Elements to a List

There are three different functions used to add elements in an existing list which are `append()`, `extend()` and `insert()`. Let us learn about them in detail.

The append() Function

The `append()` function appends a single element with the given value(any datatype) at the end of the list. We use this function in a loop for adding multiple elements. With the single `append()` function, the length of the list will increase by 1. Syntax of the `append()` function is:

```
list.append(item)
```

where, item can be any value like number, string, list, etc. Examples are:

Commands	Output

`fruits = ["apple", "banana", "cherries"]`
`fruits.append("melon")`
`print(fruits)`
	["apple", "banana", "cherries", "melon"]

`cars = ["Maruti", "Ford"]`
`cars.append("Honda")`
`cars.append("Hyundai")`
`print(cars)`
	cars = ["Maruti", "Ford", "Honda", "Hyundai"]

`#input names of 5 friends and store in list`
`friends=[]`
`for i in range(5):`
`  nm=input("Enter Friend Name: ")`
`  friends.append(nm)`
`print(friends)`
	Enter Friend Name :Amit
	Enter Friend Name: Sneha
	Enter Friend Name: Swati
	Enter Friend Name: Sudhir
	Enter Friend Name: Shashi
	["Amit", "Sneha", "Swati", "Sudhir", "Shashi"]

`#adding a list at the end of another list`
`pets = ['cat', 'dog', 'rabbit']`
`wild = ['tiger', 'lion']`
`pets.append(wild)`
`print(pets)`
	['cat', 'dog', 'rabbit', ['tiger', 'lion']]

The insert() Function

The `insert()` function is used to add a single value at a specific position in an existing list. The length of the list will increase by one. Syntax of the `insert()` function is:

```
list.insert(index, value)
```

Examples:

Commands	Output

`L = [10, 20, 30]`
`L.insert(1, 40)`
`print(L)`
	[10, 40, 20, 30]

`L.insert(0, "abc")` `print(L)`	['abc', 10, 40, 20, 30]

`a = "xyz"`
`L.insert(3, a)`
`print(L)`
	['abc', 10, 40, 'xyz', 20, 30]

`L.insert(5, ["a", "b"])` `print(L)`	['abc', 10, 40, 'xyz', 20, ['a', 'b'], 30]

The extend() Function

The `extend()` function is used to append multiple values at a time in a list. The syntax of the extend() function is:

```
list.extend(iterable value)
```

where iterable value can be a list. Examples are:

Commands	Output

`n=[1,2,3,4]`
`m=[5]`
`n.extend(m)`
`print(n)`
	[1, 2, 3, 4, 5]

`pets = ['cat', 'dog', 'rabbit']`
`wild=['tiger', 'lion']`
`pets.extend(wild)`
`print(pets)`
	['cat', 'dog', 'rabbit', 'tiger', 'lion']

Some other Functions Used with List

There are some other functions also available to use with the list which are as follows:

- **sort():** This function sorts the list in ascending or descending order. This is done "in the list itself" and works for the list with values of the same data types. For example,

```
city=["Delhi", "Mumbai", "Kolkata", "Chennai"]
city.sort() #sorts the list by default in ascending order
print(city)
```

Output: ["Chennai", "Delhi", "Kolkata", "Mumbai"]

```
city.sort(reverse=True)#sorts the list descending order
```

Output: ["Mumbai", "Kolkata", "Delhi", "Chennai"]

Searching a value in a list

Example 1

Searching a specific value and displaying its number of occurrences in a list.

```
A=[10,23,45,10,12,45,10,56]
s=int(input("enter value you want to search :"))
count=0
for i in A:
    if i==s:
        count+=1
print(s," occurs ",count," times")
```

Example 2

Input a value and search whether it exists in a list or not. If it exists then display its index number.

```
flowers=["Rose","Lily","Lotus","Sunflower"]
s=input("Input a name of a flower :")
```




```
for i in range(len(flowers)):
    if s==flowers[i]:
      print(s, " found at index number ",i)
      break
else:
print("Sorry' Not Found")
```

Tuples in Python

In the previous chapter, you have learned about list. Similar to list, a tuple is a sequence of values enclosed in parentheses and its indices start with 0. Unlike list, tuple cannot be changed which means it is immutable. In this chapter, we will discuss how to create, access and perform operations on tuples.

Creation of Tuples

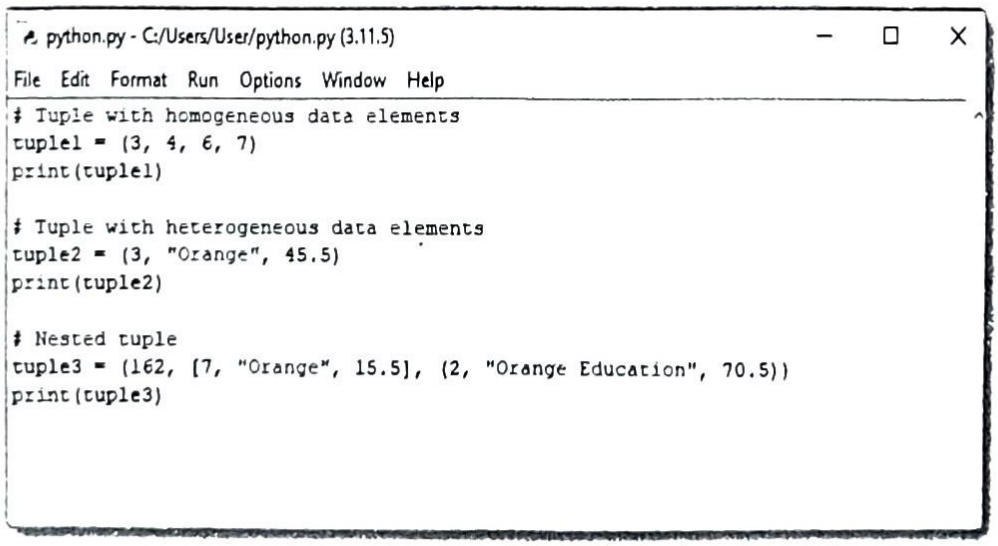
To create a tuple, write different values and separate them with comma. You can put the comma-separated values between the parentheses '()'. Parentheses (round bracket) indicate the start and end of the tuple.

Tuple contains any number of elements of heterogeneous type (integer, float, string, list, etc).

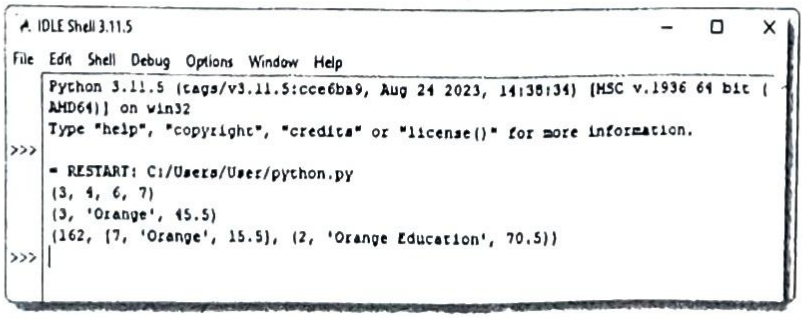
Example of tuple:

```
tup1=('sst', 'chemistry', 2010, 2022); #tuple with dissimilar items
tup2=(1, 2, 3, 4, 5 );                 #tuple with similar items
tup3=(1, 2.5, 3.6, 4, 5 );             #t 50, tup1 is treated to be of type int.
```

Program 1: To create a tuple.



On running the above program, you will get the following output:



Syntax to create a tuple:

```
tup= (value1, value2, value3, ...)
```

- To create an empty tuple: `tup= tuple()`
- To create a tuple with single element: `tup= (1,)`
- To create long tuple: `tup = (0, 1, 2, 3, 4 , 5, 6, 7, 8, 9 10, 11, 12, 13, 14, 15)`
- To create nested tuple: `tup = (1,2, (3,4))`
- To create tuple by using built-in tuple type object: `tup = tuple(<sequence>)`

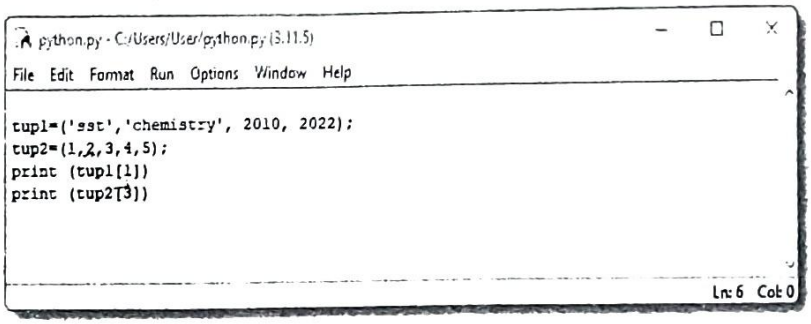
Where, sequence is an object which includes strings, lists and tuples.

```
>>>tup= tuple('orange')
>>>tup
('o', 'r', 'a', 'n', 'g','e')  #tuple tup is created from other sequence- a string
                               'orange'.
```

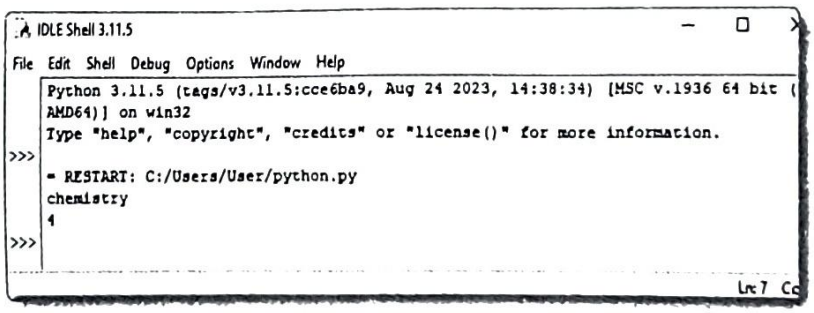
Accessing Tuples

To access values in a tuple, use the square brackets along with the index number for reference. So, you can access the tuple element just like you access a list's or string's element.

Program 2: To access a value in tuple.



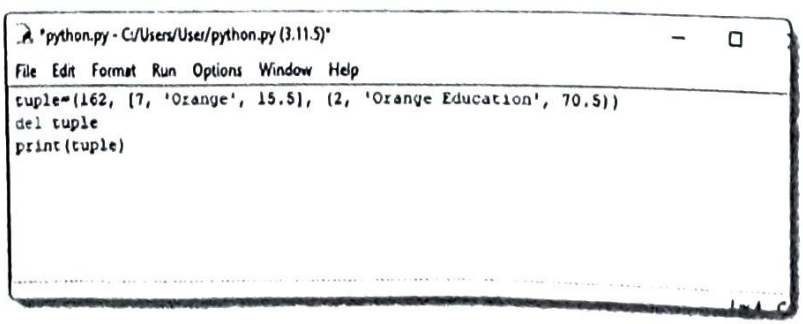
On running the above program, you will get the following output:



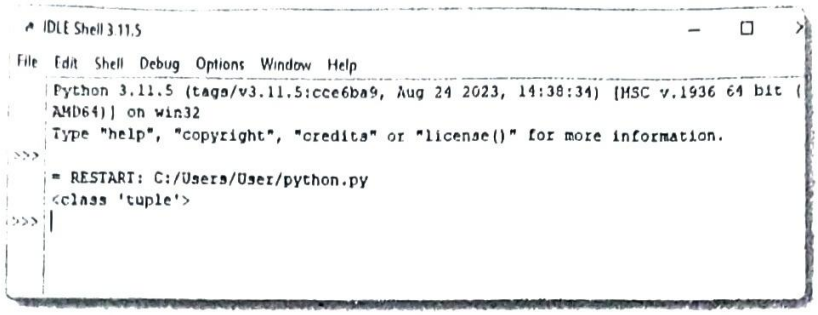
Deleting a Tuple

Elements of a tuple cannot deleted or removed, but we can delete the entire tuple by using the **del** keyword.

Program 3: To delete a tuple.



On running the above program, you will get the following output:

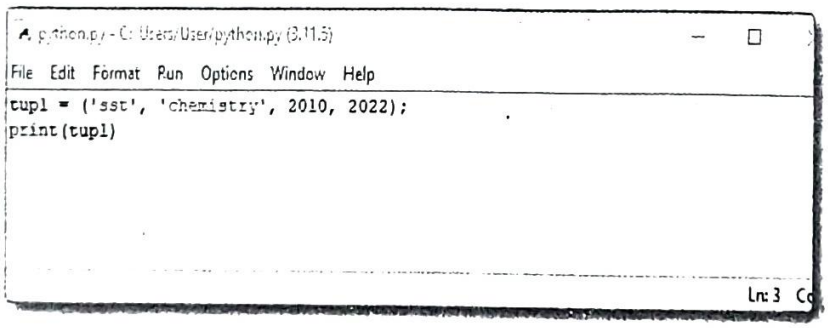


Difference between List and Tuple

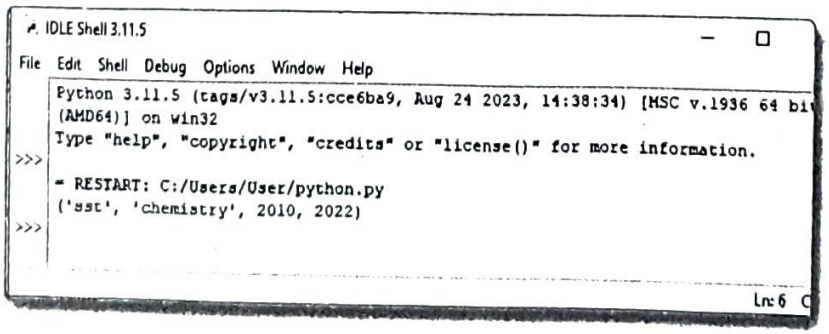
List -	Tuple
A list is a sequence of multiple values in an ordered sequence.	A tuple is a collection of objects which is ordered.
List is enclosed in square brackets [].	Tuples are enclosed in parenthesis ().
Elements of the list can be changed.	Elements of the tuple cannot be changed.
List has the variable length.	Tuple has the fixed length.
List is mutable.	Tuple is immutable.
List can hold only homogeneous values.	Tuple can hold both homogeneous and heterogeneous values.

Some More Programs

Program 4: To print the entire tuple.



On running the above program, you will get the following output:



Program 5: To use tuple method count and index.

```
File Edit Format Run Options Window Help
tup=(2,5,7,8,2,1)
print(tup.count(2))
print(tup.index(7))
```

Converting list to tuple and vice-versa

On running the above program, you will get the following output:

```
IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/User/python.py
2
2
>>>
Ln: 7 Col: 0
```

Program 6: A tuple "tup" stores (2,5,7,8,2,1) by mistake its last element is typed wrong. Write a program to correct its last element as 11.

```
File Edit Format Run Options Window Help
tup = (2, 5, 7, 8, 2, 1)
tupl = list(tup)
tupl[-1] = 11
tup = tuple(tupl)
print("Modified tuple:", tup)
Ln: 8 Col: 0
```

On running the above program, you will get the following output:

```
IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/User/python.py
Modified tuple: (2, 5, 7, 8, 2, 11)
>>>
Ln: 6 Col: 0
```

Converting List to Tuple and Vice-versa

A list can be converted into a tuple by using a built-in function tuple().

For example:

```
>>> L=[1,2,3,4]
>>> type(L)
<class 'list'>
```




```
>>> T=tuple(L)
>>> type(T)
<class 'tuple'>
```

A tuple can be converted into a list by using a built-in function list()

For example:

```
>>> L=[1,2,3,4]
>>> type(L)
<class 'list'>
>>> T=tuple(L)
>>> type(T)
<class 'tuple'>
>>> L2=list(T)
>>>L2
[1, 2, 3, 4]
>>> type(L2)
<class 'list'>
```